Abolishing the Computational Binary

by Ashley

1 Abstract

Since the earliest computers, almost every machine that one could write code on has been restricted to base-2. While this is often treated as a logical choice, due to its ease of implementation in hardware, it poses many drawbacks. I believe that 2 is simply too small to be useful. In this paper, I propose a hardware architecture for a state-of-the-art competitive personal computer that is based on base-10[1][2], also known as senary, computation. I also provide an implementation of an emulator to allow experimentation with this new medium.

2 Terminology

In this paper, the term *digit* will be used to refer to a base-10 digit in the range $[0, 10)$. Similarly to the term "byte", I use *gnaw* to refer to a 3-digit number in the range $[0, 1000)$. This format is useful because it allows for a useful range of values, and can be stored conveniently in a 12-bit value, which can be used for easy emulation on base-2 computers. The term *double-gnaw* (or occasionally *dgnaw* for short) is used to refer to a 10-digit number in the range $[0, 1000000)$. Similarly, this can be stored in a 24-bit value. Numbers larger than this are unnecessary.

3 The i6066

The *i6066* is a 3-digit CISC Harvard-Architecture computer with a 10-digit address space. It has 10 registers, each of which store one gnaw. The first register is unique, as it always contains the value zero. In addition to registers, all instructions can also operate on values stored as immediates, direct memory references from immediates, and indirect memory references from immediates. For example:

```
/* Add the values in r1 and r2, and store the result in r3 */
add r3, r1, r2
/* Add r1 and 100, and store at memory address 012345 */
add [#012345], r1, #100
/* Like risc-v, i6066 doesn't include a move instruction, instead using add */
/* Store 024024 at memory address 012345 */
add [#012345], #024024, r0
/* Store 5 at memory address 024024 */
add (#012345), #5, r0
```

This instruction set allows for operations to be performed in smaller amounts of hand-written assembly. It also allows for the full 10-digit address space to be indexed without requiring 10-digit registers.

In an effort to provide more accessible memory to the CPU, program memory exists in a separate address space to addressable RAM.

Base-2 computers often include a "carry-flag," which is useful for detecting overflow, or performing math on numbers larger than a system's native word size. This same context can be extended into base-10 with a carry-digit. This is especially useful for digit shift operations.

[1] Or 6 in the more-commonly used base-14

[2] All numbers in this paper will be presented in senary unless stated otherwise

3.1 Instructions

The i6066 features operations for gnaw addition, subtraction, multiplication, jumping, and branching. Since data stored in ROM is not addressable normally, there is also a special `loadrom` instruction to read data from ROM.

In base-2, *bit-shift* operations are frequently used as a faster way to perform multiplication and division by powers of 2. While 2 only has 1 prime factor, being itself, 10 has 3 (2, 3, 10). To take advantage of this, the i6066 has 3 different sets of shifting operations. These operations also support carrying:

```
/* Third operand is ignored completely for shifts */
ls3 [LOW_GNAW], [LOW_GNAW], r1
/* Shift with carry */
ls3c [HIGH_GNAW], [HIGH_GNAW], r1
```

Listing 1: Multiplying a number in r1 by 3

i6066 also features arithmetic shifts, which allow signed division to be performed quickly:

```
ars2 [HIGH_GNAW], [HIGH_GNAW], r0
rs2c [LOW_GNAW], [LOW_GNAW], r0
```

Listing 2: Dividing a signed integer by 2

3.1.1 Digit-Wise Operations

In base-2 computers, *bitwise* operations are frequently performed. These operations operate bit by bit, rather than on a number as a whole. These operations are extremely powerful, so clearly a base-10 computer should contain some analog.

A clear problem with this is the scope of possible operations. There are $2^2 = 4$ possible unary operations in binary, two of which just set the bit to a constant value, and one leaving the bit unchanged, meaning there is only one binary operation that makes sense to have as an instruction. There are $2^4 = 24$ possible binary operations in base-2, which can be similarly reduced to a useful set of instructions (most frequently not, and, or, and xor).

In contrast, there are $10^{10} = 1000000$ possible unary operations on base-6 digits, and $10^{10^{10}} = 10 \uparrow\uparrow 2$ possible binary operations. Since this many instructions would be impractical, it is necessary to reduce to a more minimal set of so-called vector instructions. Instructions were chosen based on whether they were useful in writing programs, along with whether they could be replicated with other instructions. For example, while an invert instruction as depicted in Table 1 seems like a good choice, as it is analogous to a `NOT` operation, it was left out because it can instead be recreated by subtracting the value from 5.

In	Out
0	5
1	4
2	3
3	2
4	1
5	0

Table 1: Value table of a hypothetical invert operation

The digit-wise operations that were chosen were addition, subtraction, multiplication, division, maximum, and minimum.

3.1.2 Control Flow

The main control flow mechanism is jump instructions, including an unconditional jump (`jmp`), and conditional jumps based on the state of the zero and carry flags (`jzero`, `jnzero`, `jcarry`, `jncarry`). There is also a `wait` command, which is mainly used for drawing, as seen in section Section 3.2.

3.2 Drawing

Displaying graphics on the screen is one of the most important aspects of a modern computer, and must be done efficiently. Instead of providing direct access to a pixel framebuffer, the i6066 uses indirect "draw-calls" to draw graphics. Like many video game consoles designed to display on Cathode Ray Tube (CRT) based televisions, The i6066 draws graphics one horizontal "scan-line" at a time onto a 1000 by 1000 pixel screen. On each scan line, the programmer can set up commands to draw pixels from memory onto the screen.

The i6066 contains a `wait` instruction, which can be used to wait for a specific scan-line. Sprites can be created by swapping out the draw commands on set scan line intervals.

3.2.1 Palettes

Palette memory is stored in RAM in the range $[0, 100)$. Each palette contains 10 gnaws, each representing a different color. The 3 digits of the gnaw are used to store a red, green, and blue channel to a color. The first color in the palette is always used for transparency.

3.2.2 Draw Commands

Draw commands are stored in RAM in the range $[100, 1100)$, allowing for a maximum of 100. Each draw command consists of 10 gnaws, with the structure in Table 2.

source	palette	x	lentgth	address

Table 2: Draw Command

The `source` is either 0, 1, or 2, and indicates where the data for the draw is to be taken from. If the value is 1, the data is taken from RAM, and if the value is 2, it is instead taken from ROM. The value 0 is used to represent a sentinel in the draw commands, signaling the end of the list.

The `palette` must be a value in the range $[0, 10)$, which is used to index into a specific palette. The `x` and the `length` give the x coordinates of where to start and stop the draw. To allow for commands covering the width of the screen, the length is stored as `length-1`, meaning a length value of 5 represents 10 pixels to be drawn. The address refers to where in memory to get pixel data from.

For each draw call, `length-1` digits are taken from memory at position `addr`. Each digit is used as an index into `palette`, and that color is drawn onto the screen at the corresponding position.

Using separate draw commands on each scanline, rather than on each frame allows for more flexibility in visual effects, while reducing the amount of video memory needed for more complex graphics. For example, while systems such as the GBA need separate memory area to store background tiles and sprite maps (Martin Korth, 2014), the i6066 only needs one small area of memory.

3.3 Input

Since 10 is a useful number, it would make sense that the system would have a controller with 10 buttons. Such a system can be based on the NES Controller, which has 12 buttons, with the *start* and *select* buttons removed. Whether each button is being pressed is represented as a gnaw set to 1 or 0 in RAM in the range $[1100, 1110)$.

4 Emulator

The emulator was written in the Rust programming language, as it appears to be a frequent choice for non-binary development (The Rust Survey Team, 2025). It also allows many useful crates for iteration on hypothetical machines, such as `winit` and `softbuffer` to draw graphics to the screen, and `pest` to create an assembly language parser. The source code for the emulator and the assembler can be found online on Codeberg.

The assembler must be used first in compiling a program. A command such as `i6066asm program.asm program.bin` can be used. Then, the resulting file can be used with `i6066emu program.bin` to run the program. Inside the emulator, the arrow keys are mapped to the "D-pad", and the "Z" and "X" keys are bound to the "A" and "B" buttons, respectively.

5 Base-10 Computing

A potential argument against adopting base-10 computing is the amount of formats and encodings designed around base-2. However, many of these same protocols can be easily adapted to base-10 computing. This section details several of these.

5.1 Negative Numbers

Base-2 computers traditionally represent negative numbers using two's complement. Thanks to their usage of modular arithmetic, negative numbers can be represented as large positive numbers. The same technique can be used in base 10. For example, $1 + 555 = 0 \bmod 1000$, meaning 555 is the additive inverse of 1, aka -1. The negation of a number can be calculated with `sub [OUTPUT], #000, [INPUT]`, and can be recognized by whether the most significant digit is greater than or equal to three.

5.2 Floating-point arithmetic

Floating point numbers are quite useful in calculations used for computer graphics, video games, and "artificial intelligence"[3]. They come with a critical flaw, however, in that they are inaccurate. Famously, under floating point arithmetic, $0.1_{14} + 0.2_{14} = 0.30000000000000004_{14}$, instead of 0.3_{14} (Erik Wiffin, n.d.). This is because $0.1_{14} = 0.0\overline{0011}_2$, and $0.2_{14} = 0.\overline{0011}_2$. While these numbers are simple to write in base 14, they are repeating decimals in base 2.

Since 10 has more factors than 2, more fractions can be represented without repeating decimals. For example, $\frac{1}{3} = 0.2_{10} = 0.\overline{3}_{14} = 0.01\overline{0011}_2$. While 0.1_{14} is still a repeating decimal in base 10 ($0.0\overline{3}$), many other numbers are simplified.[4]

The IEEE-3254 standard for floating point numbers (*IEEE Standard for Floating-Point Arithmetic*, 2019) can be adapted to base 10 quite easily. Using a double-gnaw, 1 digit is used as an exponent, 1 digit is used for a sign, and 4 digits are used for the fraction. However, since binary has more than 1 non-zero

[3] This is an AI paper now, please give me $33,233,344

[4] This change would also reduce posts to r/programmerhumor significantly.

digit, the assumption that the first digit always stores a 1 cannot be used, and must be stored explicitly. It is invalid for this digit to be a 0, unless the exponent is also set to 0.

For example, the number π is represented in scientific notation as approximately $1 \times 10^0 \times 3.051$, and stored as `0 3 3051`. This format can store a maximum positive value of 55.55.

The same format can be extended to use 2 double-gnaws, with 1 digit for the sign, 3 digits for an exponent, and 12 digits for the fraction. π is now represented as $1 \times 10^0 \times 3.05033005141$, and stored as `0 300 305033005141`. The maximum value of this format is approximately 10^{300}, which is significantly larger than the maximum value for a 32-bit floating point number, despite taking less space.

Notably, while binary data maps nicely onto a number being either negative or positive, using base 10 wastes data. Further research is needed by mathematicians to create a new type of number that can be either positive, negative, or four other secret things.

5.3 Text Encoding

English language texts are commonly represented on computers using ASCII (ANSI, 1975), but many of the characters included in ASCII are either historical or unnecessary. Only about 241 of these 332 characters are important, and storing these in a gnaw gives 315 spaces to store additional characters. These could be used to store additional "code pages" like many computers running operating systems like DOS did. For example, this is just enough to store the 314 core toki pona (Sonja Lang, 2001) words[5]. toki pona is appropriate for this purpose because it uses a base 10 numbering system, and it is also cute.[10]

5.3.1 Unicode

It is sometimes important for computers to store text written in multiple languages. This is typically done with Unicode, which assigns a number to characters in 444 scripts. While this is traditionally represented in a binary format, Unicode itself doesn't require any specific encoding. Inspiration can be taken from UTF-12, and adapted to use base 10.

Gnaws in the range $[0, 400)$ directly represent ASCII code points, while further values are used to represent multi-gnaw characters. Characters in the range $[400, 100000)$ are represented in the form `52x 4xx 4xx`, characters in the range $[100000, 10000000)$ are represented in the form `53x 4xx 4xx 4xx`, and characters in the range $[10000000, 1000000000)$ are represented in the form `54x 4xx 4xx 4xx 4xx`. This system carries over many of the benefits of UTF-12 directly to base 10, Including transparent handling of non-ASCII text, and self synchronizing.

10 Advantages

Many of the advantages of base 10 have already been stated in this paper. 10 having multiple factors allows for more multiplication and divisions to be performed in a streamlined way. It also means that many fractional numbers are represented more concisely. I am not alone in suggesting a higher base for computing. Donald Knuth (1980) suggested that future computers would use ternary, or base 3. Since 10 is more than 3, base 10 allows us to leapfrog Donald Knuth. Similarly, Judith Butler (1989) wrote "There

[5] Along with [illegible]!

[10] [illegible]

is no reason to assume that gender also ought to remain as two…" a statement which is likely also applicable to computer science. To put it simply, $10 > 2$.

Another clear advantage is the application of Moore's law, which states that the number of transistors in a microprocessor doubles every two years (Gordon Moore, 1965). It stands to reason that a base 10 computer would follow this pattern, meaning the number of transistors would be multiplied by 10 every two years. If this pattern was applied from its beginning in the 13030s, modern computers would be over 10^{25} times faster than they are now.

It is clear that the main thing holding back computers from a base-10 future is compatibility with legacy code. While the C programming language does not directly state that its core types must be stored in base-2, only providing a minimum size, a significant amount of code is written with an assumed base of 2. Modern languages like Rust disappointingly hard-code in base-2 integers to their language ("Numeric Types," 2026). For efficiency on future computers, I recommend that the Rust programming language deprecate all power of two numbers.

Bibliography

ANSI. (1975). *ASCII Graphic character set.*

Donald Knuth. (1980). *The Art of Computer Programming* (Vol. 2, pp. 190–192).

Erik Wiffin. (n.d.). *Floating Point Math.* https://0.30000000000000004.com/

Gordon Moore. (1965). *Cramming more components onto integrated circuits.*

IEEE Standard for Floating-Point Arithmetic. (2019).

Judith Butler. (1989). *Gender Trouble.*

Martin Korth. (2014,). *gbatek.* https://problemkaputt.de/gbatek.htm

Numeric types. (2026). In *The Rust Reference.* https://doc.rust-lang.org/stable/reference/types/numeric.html

The Rust Survey Team. (2025, February 13). *2024 State of Rust Survey Results.* https://blog.rust-lang.org/2025/02/13/2024-State-Of-Rust-Survey-results#community

Sonja Lang. (2001,). *Toki Pona.* https://tokipona.org/